Priorização de Testes Automatizados sob Restrição de Tempo: uma Avaliação de Heurísticas Gulosas

Andrey Lourival Andrade Garcia[1], **Igor Cecim Vilhena**[2]

[1]Centro Universitário do Estado do Pará (CESUPA)
Análise e Projeto de Algoritmos – Belém, PA, Brasil

Abstract. *Automated test prioritization aims to detect faults earlier while retaining the complete test suite. This study compares six ordering strategies in a controlled Python prototype for order processing: reference order, random order, shortest test first, total coverage, additional coverage, and additional coverage per unit of cost. The experiment uses 567 generated cases, 18 individually seeded faults, five suite sizes, and ten repetitions per configuration. Coverage, test duration, prioritization time, and fault detection are measured through actual executions. Ordering metrics are subsequently calculated using an additive model of measured test costs. For 320 tests, the cost-aware heuristic reaches a mean APFDc of 0.9850, close to the 0.9844 obtained by shortest test first. However, its median prioritization time is 60.75 milliseconds, exceeding the execution cost of these short suites. The results connect quadratic greedy work to measured growth and show why prioritization overhead must be evaluated separately from ordering effectiveness. Conclusions are limited to this small artificial benchmark and do not establish superiority on industrial systems.*

Resumo. *A priorização de testes automatizados procura antecipar a detecção de defeitos preservando a suíte completa. Este estudo compara seis estratégias em um protótipo controlado de processamento de pedidos em Python: ordem de referência, aleatória, menor duração, cobertura total, cobertura adicional e cobertura adicional por unidade de custo. O experimento utiliza 567 casos gerados, 18 defeitos inseridos individualmente, cinco tamanhos de suíte e dez repetições por configuração. Cobertura, duração, tempo de priorização e detecção são obtidos por execuções reais. As métricas das ordenações são calculadas posteriormente por um modelo aditivo dos custos medidos. Para 320 testes, o guloso sensível ao custo alcança APFDc médio de 0,9850, próximo do valor 0,9844 da ordenação por duração. Entretanto, seu tempo mediano de priorização é 60,75 milissegundos, superior ao custo de execução dessas suítes curtas. Os resultados relacionam o trabalho quadrático dos gulosos ao crescimento observado e evidenciam a necessidade de contabilizar o custo da ordenação. As conclusões se restringem ao pequeno ambiente artificial estudado.*

Palavras-chave: priorização de testes; algoritmos gulosos; análise de complexidade; APFDc; testes de regressão.

1. Introdução

Uma suíte automatizada pode conter testes com durações e capacidades de detecção diferentes. Quando o tempo disponível é limitado, a posição de cada teste influencia o

momento em que um defeito se torna observável. A priorização procura uma permutação favorável da suíte, mantendo seus elementos. Essa formulação e a comparação de critérios de cobertura aparecem em [Rothermel et al. 2001]. Seleção, minimização e priorização constituem problemas distintos, também examinados em contexto industrial por [Di Nardo et al. 2015].

Este trabalho adota o eixo E5, algoritmos gulosos, com apoio do eixo E1, análise de complexidade. Seu objetivo é comparar heurísticas simples quanto à antecipação da detecção e ao trabalho computacional necessário para produzir a ordem. O objeto é o MiniPedidos, um protótipo controlado de desconto, frete, estoque e fechamento de pedidos. Sua pequena escala permite inspecionar o procedimento, mas limita a transferência dos resultados para aplicações reais.

As perguntas de pesquisa são: **QP1**, como as estratégias diferem em detecção antecipada quando os testes têm durações distintas? **QP2**, o crescimento do custo dos priorizadores acompanha a análise assintótica? **QP3**, o benefício persiste quando se inclui o tempo de calcular a ordem? A contribuição é um protocolo reproduzível e uma análise de seus compromissos, sem reivindicar um algoritmo novo.

2. Fundamentação teórica e trabalhos relacionados

O critério total privilegia testes que cobrem muitos elementos. O adicional considera somente elementos ainda não cobertos na rodada corrente. [Li et al. 2007] analisam estratégias gulosas e de busca e mostram que objetivos diferentes podem favorecer ordens distintas. A comparação é pertinente para E5, mas antecipar cobertura não garante antecipar defeitos.

O trabalho de [Chvátal 1979] estabelece o limite harmônico $H(d)$ para o custo de uma cobertura de conjuntos obtida por escolha gulosa, com d igual ao maior tamanho de um conjunto. A razão entre novos elementos e custo inspira uma das heurísticas deste trabalho. A garantia refere-se à cobertura de custo mínimo; ela não é reivindicada para APFD, APFDc ou para a permutação completa aqui produzida.

[Elbaum et al. 2001] incorporam custos e severidades à avaliação de priorização, enquanto [Walcott et al. 2006] exploram uma abordagem genética sensível ao tempo. A revisão de [Ahmed et al. 2022] organiza métodos e métricas que consideram custo e valor. Neste projeto, a severidade é uniforme: não existem dados que permitam atribuir pesos realistas aos defeitos artificiais.

Entre os trabalhos recentes, [Li et al. 2022] reduzem o esforço do guloso adicional por aceleração e [Zhang et al. 2022] investigam atenção parcial. Ambos delimitam alternativas relevantes, mas a reprodução de aprendizado ou de AGA ampliaria o recorte. A comparação implementada utiliza critérios transparentes, cuja execução pode ser contada diretamente. A Tabela 1 sintetiza diferenças que orientam o protocolo.

A família de estudos de [Elbaum et al. 2002] situa a investigação empírica de priorização. [Inozemtseva and Holmes 2014] observam que a relação entre cobertura e eficácia enfraquece ao controlar o tamanho das suítes. [Just et al. 2014] estudam a relação entre mutantes e defeitos reais, mas sua evidência não transforma qualquer conjunto artificial em substituto universal de falhas naturais. Por isso, o protocolo mantém fixos os testes de cada comparação e explicita a origem dos defeitos.

Tabela 1. Comparação crítica das linhas de trabalho selecionadas.

Referência	Foco	Consequência para este projeto
Li et al. (2007)	Gulosos e busca por cobertura	Comparar total e adicional, sem tratar cobertura como detecção.
Walcott et al. (2006)	Tempo e busca genética	Medir orçamento temporal, sem reproduzir o método genético.
Li et al. (2022)	Aceleração do adicional	Contabilizar o custo da própria priorização.
Zhang et al. (2022)	Atenção parcial	Reconhecer alternativas recentes fora do recorte.
Inozemtseva e Holmes (2014)	Cobertura e eficácia	Avaliar detecção separadamente.

A lacuna adotada é local e didática: verificar se o benefício de reordenar funções muito curtas compensa o custo do priorizador. Os resultados de outros sistemas não resolvem essa questão para o MiniPedidos. A matriz anexa contém 12 artigos, três de 2022 e o clássico de 1979. Os níveis de consulta são discriminados, e referências acessadas apenas pelo resumo não sustentam detalhes quantitativos.

3. Metodologia

3.1. Formulação e critérios de ordenação

A entrada é uma suíte $T = \{t_1, \ldots, t_n\}$, a ordem de referência dos seus identificadores, um conjunto de linhas cobertas $C_i \subseteq U$ por teste e uma duração estimada $c_i > 0$. Denota-se por $m = |U|$ o número de linhas distintas observadas. A saída é uma permutação π dos n testes. O orçamento B é aplicado à avaliação do prefixo completo executável: $k_B = \max\{k : \sum_{j=1}^{k} c_{\pi(j)} \leq B\}$. Um teste que não cabe não é saltado para executar outro.

Cada estratégia produz a ordem completa antes dessa avaliação. O conjunto de defeitos detectados por um teste é desconhecido para o priorizador; sua matriz é usada somente depois, para avaliar a ordem. Assim, nenhuma estratégia recebe antecipadamente a resposta que deveria descobrir.

São comparadas seis estratégias: *referência*, que copia a ordem inicial; *aleatória*, que embaralha com semente fixa; *menor duração*, por c_i crescente; *total*, por $|C_i|$ decrescente; *adicional*, por $|C_i \cap V|$; e *adicional/custo*, por $|C_i \cap V|/c_i$. Aqui V é a cobertura ainda não obtida na rodada. Empates dos métodos determinísticos são resolvidos pelo identificador lexicograficamente menor. A ordem inicial é arbitrária e fixa, não um histórico industrial.

Nos dois últimos métodos, começa-se com $V = U$. Após escolher t_i, remove-se C_i de V e remove-se t_i dos candidatos. Se todos os candidatos têm ganho zero e $V \neq U$, restaura-se $V = U$ e recalculam-se os ganhos uma vez. Esse reinício também atende ao caso em que as linhas restantes só eram cobertas por testes já escolhidos. Testes de ganho zero continuam na saída, inclusive quando U é vazio.

3.2. Corretude e complexidade

O invariante do laço é: os identificadores já inseridos são distintos, pertencem à entrada e, juntamente com os candidatos restantes, particionam T. A inicialização satisfaz o invariante. Cada iteração seleciona um candidato, acrescenta-o uma única vez e o remove do conjunto restante; o reinício de cobertura não modifica essa partição. Como o número de candidatos diminui em uma unidade, o laço termina após n escolhas. Logo, a saída é uma permutação completa. A prova assegura validade da saída, e não otimalidade da detecção.

Na implementação, um ganho percorre explicitamente C_i e consulta a pertinência a V. Com operações esperadas constantes em conjuntos por hash, há no máximo duas varreduras de candidatos por escolha. Portanto, o número de consultas é limitado por $2m \sum_{r=1}^{n} r = mn(n+1)$, além da construção de U. Para $m \geq 1$, o tempo é $O(n^2 m)$; incluindo cobertura vazia, a expressão é $O(n^2(m+1))$. Não há ordenação de candidatos a cada rodada: a comparação da tupla de pontuação e identificador resolve o empate.

Tabela 2. Complexidade da implementação com cobertura já materializada.

Estratégia	Tempo	Espaço auxiliar
Referência	$O(n)$	$O(n)$
Aleatória	$O(n)$	$O(n)$
Menor duração	$O(n \log n)$	$O(n)$
Cobertura total	$O(n \log n)$	$O(n)$
Adicional	$O(n^2(m+1))$	$O(n+m)$
Adicional/custo	$O(n^2(m+1))$	$O(n+m)$

A entrada ocupa até $O(nm+n)$, não incluído na coluna auxiliar. O comprimento de um conjunto Python já está armazenado, justificando $O(n \log n)$ para total neste estágio. Construir a matriz a partir da execução é um custo anterior, que depende do programa. Isso explica a diferença em relação a análises que incluem uma varredura inicial de nm posições. O modelo assume identificadores de tamanho limitado e custos numéricos de tamanho fixo.

3.3. Objeto, dados e procedimento experimental

O MiniPedidos opera com valores monetários inteiros em centavos, evitando arredondamento monetário em ponto flutuante. O gerador cria 120 casos de desconto, 160 de frete, 42 de reserva de estoque e 245 de fechamento, totalizando 567. Há valores válidos, limites e entradas inválidas; pedidos de tamanhos distintos criam durações diferentes. Um oráculo separado calcula as saídas esperadas segundo as regras declaradas em `docs/especificacao.md`.

São inseridos 18 defeitos isolados, alterando, por exemplo, comparações de fronteira, constantes, composição do total e arredondamento do peso. Cada versão contém exatamente uma alteração catalogada. Todos os testes passam no programa-base, e cada defeito é detectado por pelo menos um caso da suíte completa. A lista foi construída para este estudo: não é uma amostra de defeitos históricos.

A cobertura de linhas é coletada com rastreamento restrito ao arquivo do objeto. A medição temporal ocorre sem esse rastreamento, após cinco chamadas de aquecimento por caso. Para cada teste, são cronometrados dez lotes de 50 chamadas, com ordem dos casos embaralhada a cada repetição. O custo individual é a mediana das durações por chamada. Não há espera artificial para aumentar custos.

Para cada uma de dez sementes de amostragem, um embaralhamento dos 567 casos fornece prefixos de $20, 40, 80, 160$ e 320 casos. Essas são instâncias experimentais: todas as estratégias recebem os mesmos casos em cada comparação e devolvem todos eles. A amostragem para variar n não é uma técnica de seleção de testes. As instâncias de tamanhos diferentes se sobrepõem intencionalmente.

O desenho contém $5 \times 10 \times 6 = 300$ configurações avaliadas. O tempo de cada priorizador é a mediana de três chamadas internas; a memória auxiliar é medida em chamada separada por `tracemalloc`, com a entrada previamente alocada. Contam-se avaliações de pontuação e consultas de pertinência. Tempos, ordens, sementes, matriz de defeitos e ambiente ficam registrados em arquivos abertos. As comparações apresentam médias, desvios-padrão e medianas, sem atribuir significância estatística a observações dependentes.

3.4. Métricas e interpretação temporal

Seja q o número de defeitos detectáveis pela instância, TF_f a posição do primeiro teste que detecta f e $C = \sum_i c_i$. Usa-se

$$\mathrm{APFD} = 1 - \frac{\sum_{f=1}^{q} TF_f}{nq} + \frac{1}{2n}. \tag{1}$$

Para incorporar durações, com severidade uniforme, define-se s_f como o instante de início do primeiro teste detector e d_f como sua duração:

$$\mathrm{APFDc} = 1 - \frac{\sum_{f=1}^{q} (s_f + d_f/2)}{qC}. \tag{2}$$

A detecção real só fica disponível ao terminar o teste. Por isso, também é registrada a área da curva em degraus $\mathrm{AUC}_{fim} = 1 - \sum_f (s_f + d_f)/(qC)$, que não usa a convenção de ponto médio do APFDc. O tempo até a primeira detecção é $\min_f(s_f + d_f)$. Somar o tempo de priorização P produz uma estimativa de feedback total $P + \min_f(s_f + d_f)$.

Registram-se ainda as frações de defeitos detectados com 10%, 25%, 50% e 75% de C e a cobertura atingida com 25%. Os orçamentos desses indicadores excluem P; o feedback total o inclui explicitamente. O denominador q é registrado por instância e não é necessariamente 18 em uma subamostra. A comparação entre estratégias permanece pareada sobre o mesmo conjunto detectável.

Custos, cobertura e detecção das versões defeituosas resultam de execuções reais. Entretanto, os tempos de cada permutação são **simulados por soma dos custos medianos do programa-base**. Não se cronometra um pipeline completo para cada ordem. O modelo desconsidera inicialização de processos, E/S, concorrência, dependência entre testes e variação da duração causada pelo defeito. Essa distinção impede interpretar o APFDc como ganho observado em integração contínua.

4. Resultados e discussão

O experimento foi executado em 15 de outubro de 2026 com Python 3.12.14, Linux x86_64 e processador identificado como AMD EPYC 9V74. O ambiente expôs nove CPUs lógicas, mas o procedimento foi sequencial, sem fixar afinidade. O relógio monotônico informou resolução de 1 ns; resolução não significa precisão efetiva de 1 ns. A suíte completa observou 40 linhas do objeto. Os arquivos de ambiente e resultados preservam a descrição exata e o hash do programa-base.

4.1. QP1: antecipação da detecção

A Tabela 3 apresenta as dez repetições de $n = 320$. O método adicional/custo atingiu APFDc médio de 0,9850 e menor duração atingiu 0,9844. A proximidade não sustenta uma superioridade robusta do guloso. Em APFD, que ignora a duração, os valores foram 0,9413 e 0,7663, respectivamente. Portanto, mudar a unidade do eixo temporal altera a avaliação relativa das ordens.

Tabela 3. Detecção e custo em $n = 320$; média $\pm$ desvio-padrão do APFDc.

Estratégia	APFDc	Falhas a 25%	P mediano (ms)	Aux. (KiB)
Referência	$0,8752 \pm 0,0275$	79,82%	0,0026	2,85
Aleatória	$0,8986 \pm 0,0460$	85,75%	0,0577	5,86
Menor duração	$0,9844 \pm 0,0016$	100,00%	0,0599	10,49
Total	$0,7053 \pm 0,0308$	62,53%	0,0911	17,77
Adicional	$0,9154 \pm 0,0185$	91,41%	57,2595	50,15
Adicional/custo	$0,9850 \pm 0,0054$	98,27%	60,7477	50,40

A cobertura total apresentou o menor APFDc desta comparação. No protótipo, os testes de pedidos extensos atravessam muitas linhas e podem atrasar a execução de verificações curtas. Esse mecanismo é uma interpretação compatível com os dados e com o gerador, não uma demonstração causal isolada. O adicional reduz redundância na cobertura, enquanto a divisão por duração favorece ganhos rápidos. A ordenação apenas por duração já aproveita fortemente a estrutura deste conjunto artificial.

Na Figura 1, o comportamento depende do tamanho da instância. Comparar tamanhos exige cautela: muda também o conjunto detectável. Em $n = 320$, o número médio de defeitos detectáveis foi 17,3, e não 18 em todas as amostras. Valores de 100% no orçamento indicam todos os defeitos detectáveis daquela instância. A conclusão sobre um algoritmo não pode ser baseada apenas em sua maior média em uma tabela.

4.2. QP2: crescimento observado e limites teóricos

A Figura 2 mostra o tempo mediano de priorização. O ajuste exploratório $t(n) = an^b$, por regressão linear em escala logarítmica dos cinco pontos medianos, produziu $b = 1,996$ para ambos os gulosos adicionais, com $R^2 > 0,999$. Como m permanece limitado a 40 neste objeto, o crescimento é compatível com o termo quadrático em n. O ajuste descreve somente o intervalo observado; não constitui prova de complexidade nem valida o comportamento quando m também cresce.

O número de consultas aos conjuntos, mostrado na Figura 3, oferece evidência independente de ruído do relógio. A curva de referência n^2 é escalada ao primeiro ponto

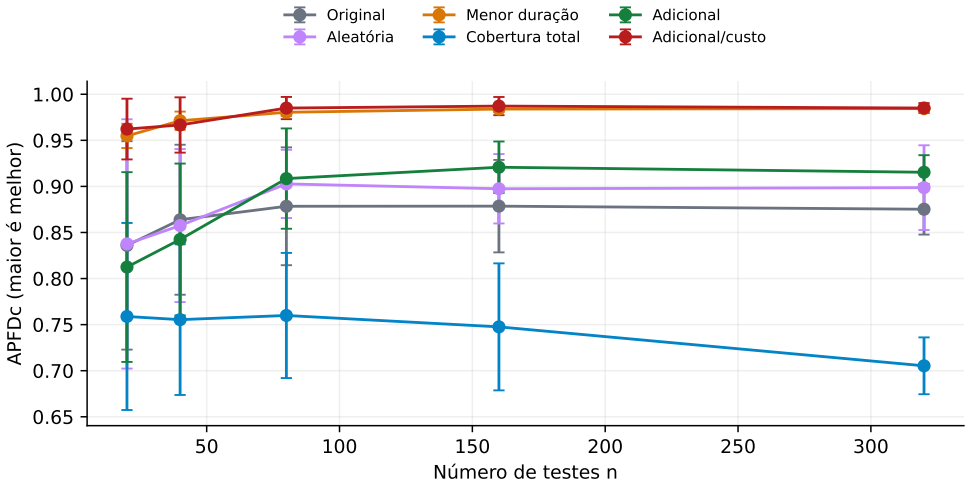


Figura 1. APFDc médio por tamanho. As barras representam um desvio-padrão, não intervalos de confiança.

e serve apenas para comparar crescimento. Reinícios e tamanhos variáveis de C_i alteram as constantes. Dividir pelo custo acrescenta operações, mas não muda o limite assintótico adotado.

Os expoentes ajustados das estratégias referência, aleatória, menor duração e total foram 0,355, 0,672, 1,032 e 1,140. Os dois primeiros são inferiores ao limite linear, o que é plausível no intervalo pequeno: custos fixos de chamadas, construção do gerador aleatório e medição afetam a curva. Os próximos de um não distinguem crescimento linear de $n \log n$ nesse domínio. Não se substitui a análise do código por um expoente ajustado a cinco pontos.

O pico mediano auxiliar em $n = 320$ ficou próximo de 50 KiB nos gulosos, acima dos métodos de ordenação simples. A medição não representa memória residente total, não inclui a matriz de entrada previamente alocada e não identifica sozinha uma classe assintótica. Ela complementa a derivação $O(n + m)$ e permite verificar o custo concreto da implementação.

4.3. QP3: custo para obter feedback

Ao somar priorização e primeira detecção, as medianas em $n = 320$ foram 0,0047 ms para referência, 0,0592 ms para aleatória, 0,0601 ms para menor duração, 0,1223 ms para total, 57,2813 ms para adicional e 60,7485 ms para adicional/custo. O excelente APFDc do último não compensou seu cálculo nesse critério. Esses valores são medianas das somas por instância; não foram obtidos somando medianas independentes.

O achado decorre do regime estudado: testes de funções curtas e um priorizador que recalcula ganhos repetidamente. Não significa que priorização seja dispensável em sistemas com testes demorados. O custo de coletar cobertura e durações também não entra em P, pois é tratado como informação anterior disponível. Cobrá-lo em cada ciclo aumentaria o custo total. Reutilização entre versões e aceleração do adicional são extensões relevantes, mas não foram avaliadas.

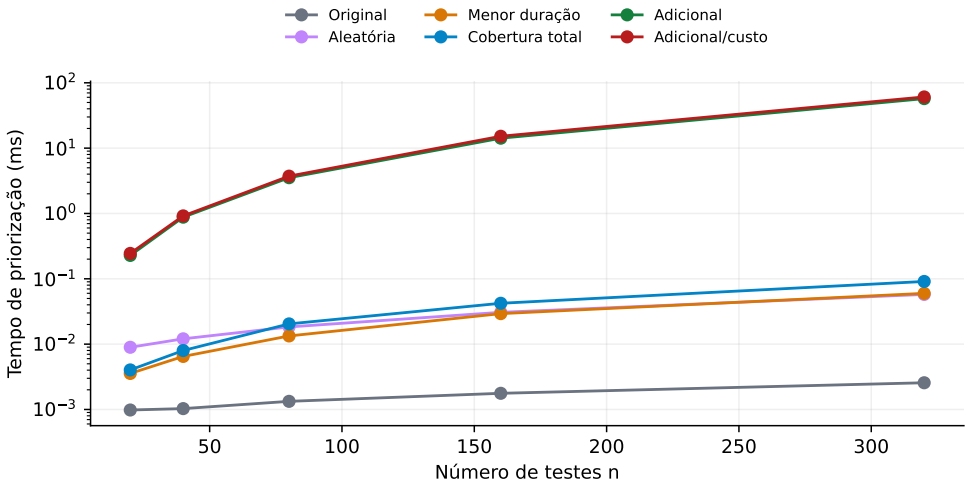


Figura 2. Tempo mediano de priorização; eixo vertical logarítmico.

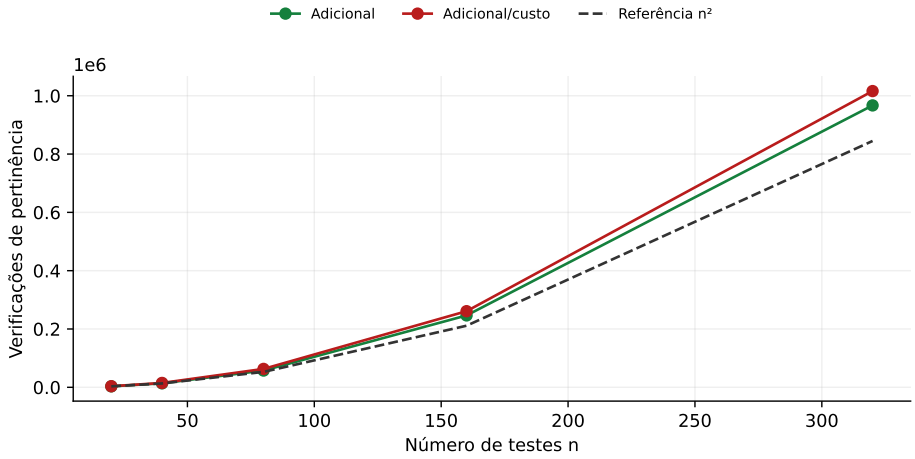


Figura 3. Consultas de pertinência e referência quadrática com escala ajustada.

4.4. Ameaças à validade

Construção: os defeitos são alterações planejadas, e casos e oráculo compartilham uma especificação. Erros correlacionados entre eles são possíveis. Seis testes automatizados do método verificam exemplos de métricas, permutações, término com ganho zero e casos-base; esses controles reduzem erros de implementação, sem provar que o modelo de defeitos é representativo.

Interna: o ambiente compartilhado, o coletor de memória, o interpretador e a escala de microssegundos introduzem ruído. Cobertura e memória foram medidas separadamente do tempo, e lotes reduziram parte do custo fixo. Cada configuração foi repetida, mas a ordem fixa de avaliação das estratégias ainda permite efeitos de deriva do ambiente. Custos são medianas únicas reutilizadas na simulação, não novas amostras a cada permutação.

Externa: há somente um programa pequeno, sem banco de dados, rede, de-

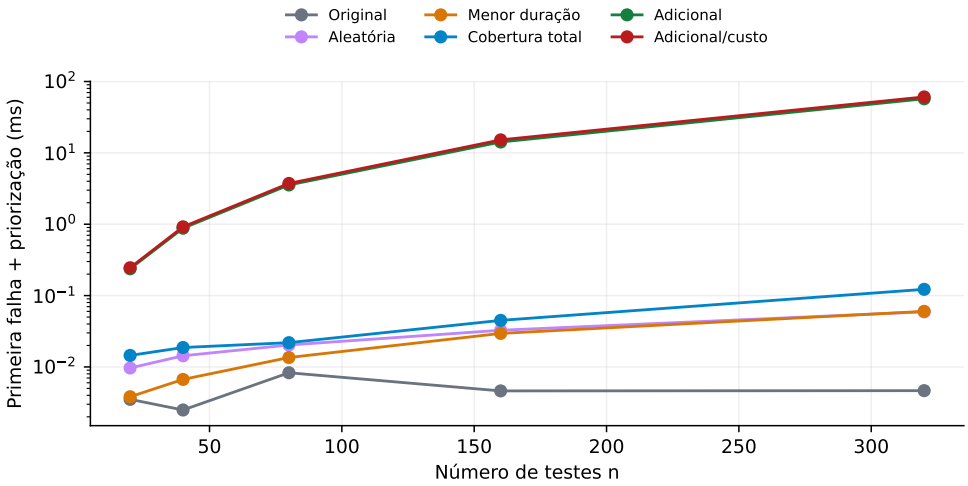


Figura 4. Estimativa de tempo até o primeiro defeito, incluindo priorização.

pendências de execução ou defeitos históricos. O sorteio de subconjuntos não cria dez sistemas independentes. **Conclusão:** as médias descrevem este experimento; não foram realizados testes de hipótese nem uma análise de poder. Uma avaliação industrial, com diferentes sistemas, durações e histórias de evolução, é necessária antes de recomendar uma estratégia geral.

5. Conclusão

A comparação preservou todos os testes de cada instância e relacionou heurísticas gulosas ao custo da implementação. No ambiente controlado, adicional/custo e menor duração apresentaram APFDc semelhante, enquanto a ordem de referência obteve o menor tempo até o primeiro defeito quando se incluiu a priorização. Assim, eficácia da ordem e custo para produzi-la respondem a questões diferentes.

O crescimento dos gulosos foi compatível com $O(n^2m)$ para m limitado. Como extensões, propõem-se sistemas existentes, defeitos históricos e redução dos recálculos. O pacote inclui código e dados; os autores deverão acrescentar a URL do repositório público antes da entrega.

Uso de IA e estado de revisão. Houve apoio das ferramentas de IA ChatGPT e Claude na busca e conferência de fontes, elaboração desta minuta, implementação, depuração, execução e interpretação inicial dos experimentos. Os resultados apresentados foram produzidos pelo código disponibilizado. Os autores ainda devem revisar as afirmações, realizar suas próprias leituras, repetir o experimento e assumir responsabilidade pelo texto. Os contatos dos autores aguardam confirmação.

Referências

Ahmed, F. S., Majeed, A., Khan, T. A., and Bhatti, S. N. (2022). Value-based cost-cognizant test case prioritization for regression testing. *PLOS ONE*, 17(5):e0264972. DOI: `https://doi.org/10.1371/journal.pone.0264972`.

Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235. DOI: https://doi.org/10.1287/moor.4.3.233.

Di Nardo, D., Alshahwan, N., Briand, L., and Labiche, Y. (2015). Coverage-based regression test case selection, minimization and prioritization: A case study on an industrial system. *Software Testing, Verification and Reliability*, 25(4):371–396. DOI: https://doi.org/10.1002/stvr.1572.

Elbaum, S., Malishevsky, A. G., and Rothermel, G. (2001). Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 329–338. IEEE. DOI: https://doi.org/10.1109/ICSE.2001.919106.

Elbaum, S., Malishevsky, A. G., and Rothermel, G. (2002). Test case prioritization: A family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2):159–182. DOI: https://doi.org/10.1109/32.988497.

Inozemtseva, L. and Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 435–445. ACM. DOI: https://doi.org/10.1145/2568225.2568271.

Just, R., Jalali, D., Inozemtseva, L., Ernst, M. D., Holmes, R., and Fraser, G. (2014). Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM. DOI: https://doi.org/10.1145/2635868.2635929.

Li, F., Zhou, J., Li, Y., Hao, D., and Zhang, L. (2022). AGA: An accelerated greedy additional algorithm for test case prioritization. *IEEE Transactions on Software Engineering*, 48(12):5102–5119. DOI: https://doi.org/10.1109/TSE.2021.3137929.

Li, Z., Harman, M., and Hierons, R. M. (2007). Search algorithms for regression test case prioritization. *IEEE Transactions on Software Engineering*, 33(4):225–237. DOI: https://doi.org/10.1109/TSE.2007.38.

Rothermel, G., Untch, R. H., Chu, C., and Harrold, M. J. (2001). Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948. DOI: https://doi.org/10.1109/32.962562.

Walcott, K. R., Soffa, M. L., Kapfhammer, G. M., and Roos, R. S. (2006). TimeAware test suite prioritization. In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, pages 1–12. ACM. DOI: https://doi.org/10.1145/1146238.1146240.

Zhang, Q., Fang, C., Sun, W., Yu, S., Xu, Y., and Liu, Y. (2022). Test case prioritization using partial attention. *Journal of Systems and Software*, 192:111419. DOI: https://doi.org/10.1016/j.jss.2022.111419.